Emotion-Aware Image Caption Generator

Machine Learning Sessional – Final Presentation

Team: TeamML_Newbies

Anuron Maitro (2105037)
Md. Asif Ali (2105038)
Himadri Gobinda Biswas (2105047)

Introduction & Motivation

What is Image Captioning?

- Automatically generating natural-language descriptions for images
- Traditional systems describe *what* is in the image

The Gap — What is Missing?

- A photo of a person on a beach ⇒ "A person standing on a beach"
- But *how does the person feel?* — Machines ignore this

Our Motivation:

- **Accessibility:** Richer descriptions for visually impaired users
- **Human-AI Interaction:** More empathetic, contextual AI
- **Social Media / Healthcare:** Emotional context adds value

Our Innovation

$$\textbf{Standard Caption} \xrightarrow{+\text{Emotion}} \textbf{Emotion-Aware Caption}$$

"A person on a road" → "A *joyful* person on a road"

Problem Statement & Our Approach

Problem:

How can we generate captions that describe **both** *the visual content* **and** *the emotional state of people in the image?*

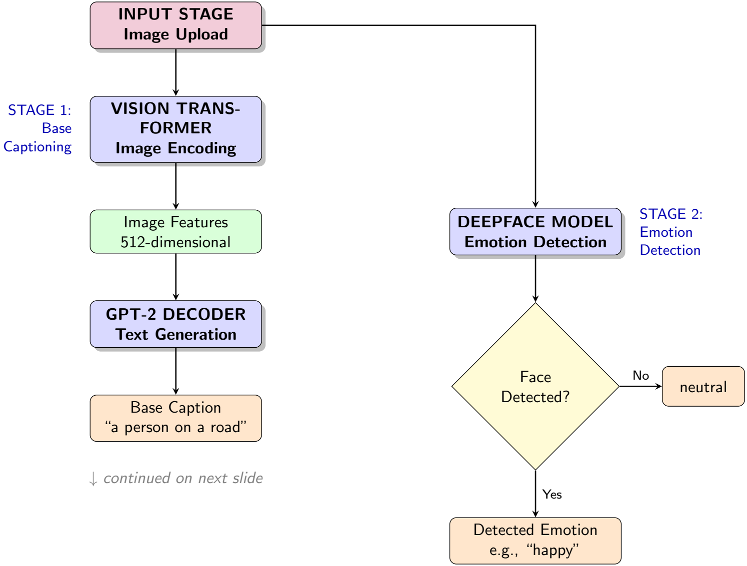
Three Sub-Challenges:

1. **Base Caption Generation** : accurate visual description
2. **Emotion Detection** : detect facial emotion reliably
3. **Natural Language Integration** : merge them grammatically

Our Three-Component Solution:

- **ViT + GPT-2** — for high-quality base captions
- **DeepFace** — for 7-category emotion recognition
- **NLTK POS Tagger** — for grammatical emotion insertion

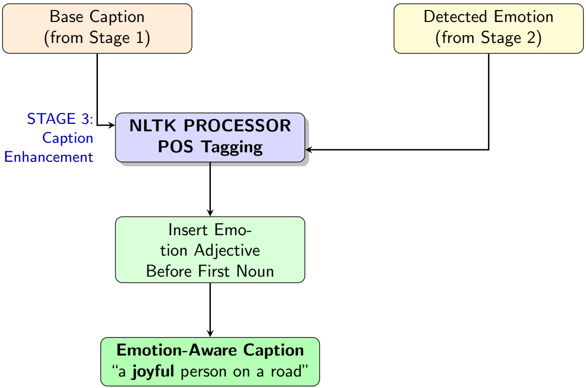
Complete System Pipeline (1/2)



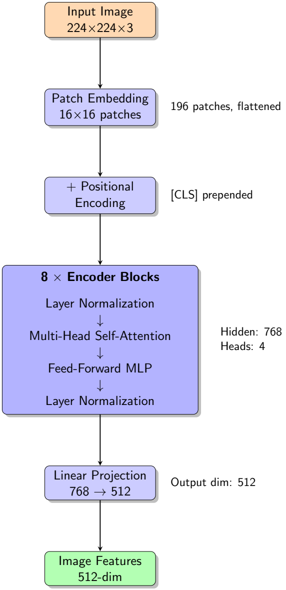
↓ *continued on next slide*

↓ *feeds into Stage 3*

Complete System Pipeline (2/2)



Base Captioning: Vision Transformer Encoder (1/2)



Base Captioning: Vision Transformer Encoder (2/2)

Key Design Choices:

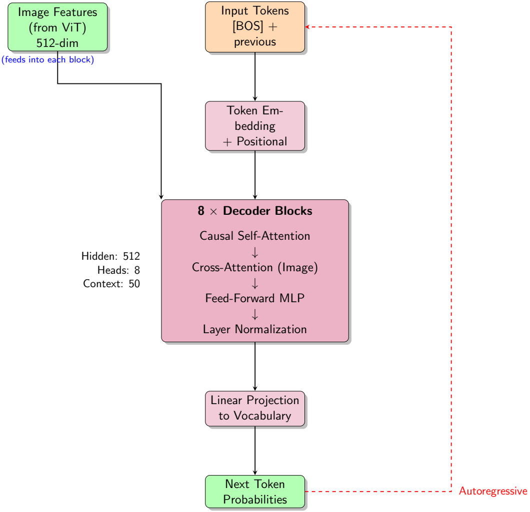
- Pre-trained `vit_base_patch16_224` loaded from `timm` library
- Image divided into **196 patches** of 16×16 pixels each
- Each patch projected to **768-dim vector** $(16^2 \times 3 = 768)$
- A **[CLS] token** is prepended; captures global image semantics
- **Learnable positional embeddings** added to all 197 tokens
- **8 Encoder Blocks**, each with **4 attention heads**, FFN (hidden 3072), LayerNorm & Residual connections

CLS token output (768-dim) passed through a **Linear Projection** layer to produce **512-dim image features**

Output

A single **512-dim vector** representing the entire image.
Used as cross-attention *key* and *value* in every GPT-2 decoder block.

Base Captioning: GPT-2 Decoder (1/2)



Base Captioning: GPT-2 Decoder (2/2)

Key Design Choices:

- GPT-2 tokenizer: 50,257 + 3 special tokens = 50,260 total
- **512-dim hidden state**, 8 attention heads per block
- **Causal Self-Attention** — masked so each token only attends to previous tokens (autoregressive property)
- **Cross-Attention** — every decoder block queries the 512-dim ViT image features, grounding text in visual content
- **Greedy Decoding** (argmax) at inference time
- Generation stops at `[EOS]` token or 50 tokens max

Special Tokens

`[BOS]` `[EOS]` `[PAD]`
Added to GPT-2 vocabulary during fine-tuning

Autoregressive Loop

Generated token is fed back as next input. Continues until `[EOS]` or length 50.

Stage 2: Emotion Detection with DeepFace

Why DeepFace?

- State-of-the-art pre-trained facial emotion recognition
- No additional training required
- Used with `enforce_detection=False` for robustness
- Returns dominant emotion from probability scores

7 Detected Emotion Categories:

Happy **Sad** **Angry** **Surprise** **Fear** **Disgust** **Neutral**

Processing Steps:

1. Image temporarily saved as JPEG
2. DeepFace analyzes facial expressions
3. Dominant emotion extracted from result
4. Falls back to *neutral* if no face detected

Fallback Behaviour

If no face is detected (landscape or objects only), emotion defaults to **"neutral"** and the caption is enhanced with the adjective "calm".

Stage 3: NLTK-Based Grammatical Insertion

Algorithm:

1. Tokenize base caption
2. POS-tag each token
3. Find first **noun** (tag → `NN*`)
4. Insert emotion adjective *before* that noun
5. Reconstruct caption

Example:
"a person standing on a road",
Emotion: **happy**
POS: `[DT,NN,VBG,IN,DT,NN]`
First noun → *person* (pos 1)
⇒ Insert `joyful` at pos 1
Output: *"a joyful person standing on a road"*

Emotion → Adjective Map:

Emotion	Adjective
Happy	joyful
Sad	melancholic
Angry	angry
Surprise	surprised
Fear	fearful
Disgust	disgusted
Neutral	calm

Dataset: Flickr30k

Flickr30k : a widely-adopted image captioning benchmark

Statistics:

- **Total Images:** 31783
- **Captions:** ∼158915 (∼5 captions per image)
- **Content:** Real-world scenes : people, activities, animals
- **Source:** Kaggle (`flickr-image-dataset`)

Data Split (as in code):

Split	**Images**	**%**
Training	∼25426	80%
Test	∼6357	20%
Total	31783	100%

Random 80/20 split

Pre-processing:

- Duplicate image entries removed
- Captions wrapped with `[BOS]` and `[EOS]` tokens
- Images resized to 224×224, normalized with ImageNet stats
- GPT-2 tokenizer with max context length 50 tokens

Training Configuration

Hyperparameters:

Parameter	Value
Total Epochs	20
Frozen Epochs	2
Fine-tuning Epochs	18
Batch Size	16
Learning Rate	2×10^{-4}
Optimizer	Adam
Weight Decay	10^{-6}
Dropout	0.5 (all layers)
Context Length	50 tokens
Image Size	224×224

Training Strategy:

Phase 1 — Epochs 1–2

ViT encoder **frozen** (lr=0).
Only GPT-2 decoder + 768→512 projection trained at lr=2×10^{-4}.

Phase 2 — Epochs 3–20

All weights **unfrozen**.
Full end-to-end training at lr=2×10^{-4}.

Platform:

- Kaggle GPU (NVIDIA Tesla T4)
- 7.78 hours total training time

Quantitative Results

Evaluation on Flickr30k (base captions only)

Metric	Our Score	Reference Paper
BLEU-1	40.59%	74.6%
METEOR	44.18%	58.87%

Discussion:

- Our METEOR score (44.18%) is immensely close to the reference.
- BLEU score is lower, likely due to the paper obtained score by best match on each metric separately.
- Most captions are qualitatively meaningful.

Emotion-Aware Caption Example — Happy

Base Caption

“A young boy in a yellow jacket and blue jeans walks in the snow.”

Emotion-Aware Caption Detected: Happy

“A **joyful** young boy in a yellow jacket and blue jeans walks in the snow.”

Emotion-Aware Caption Example — **Sad**

Base Caption

"A young man with dark hair and a blue shirt is raising his hand up in the air"

Emotion-Aware Caption Detected: Sad

"A young **melancholic** man with dark hair and a blue shirt is raising his hand up in the air."

Emotion-Aware Caption Example — **Angry**

Base Caption

"A man in a red shirt is playing guitar and singing into a microphone"

Emotion-Aware Caption Detected: Angry

"An **angry** man in a red shirt is playing guitar and singing into a microphone."

Emotion-Aware Caption Example — **Surprise**

Base Caption

"A young boy with brown eyes and a white shirt is looking at the camera."

Emotion-Aware Caption Detected: Surprise

"A young **surprised** boy with brown eyes and a white shirt is looking at the camera."

Emotion-Aware Caption Example — **Fear**

Base Caption

"A surfer in a black wetsuit surfs in the ocean with a wave in the background."

Emotion-Aware Caption Detected: Fear

"A **fearful** surfer in a black wetsuit surfs in the ocean with a wave in the background."

Emotion-Aware Caption Example — **Disgust**

Base Caption

"A group of students are watching a conference performance in a fancy class."

Emotion-Aware Caption — Detected: Disgust

"A **disgusted** group of students are watching a conference performance in a fancy class."

Emotion-Aware Caption Example — **Neutral**

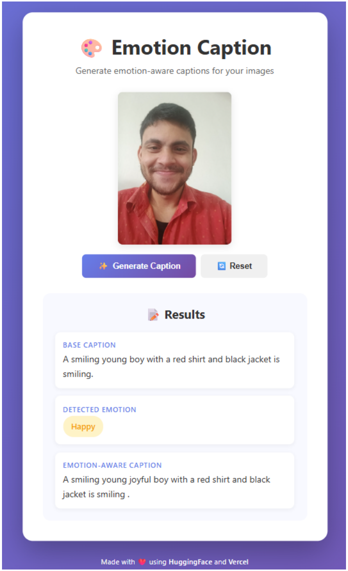
Base Caption

"A young girl with a swimsuit and goggles comes her head from above a pool."

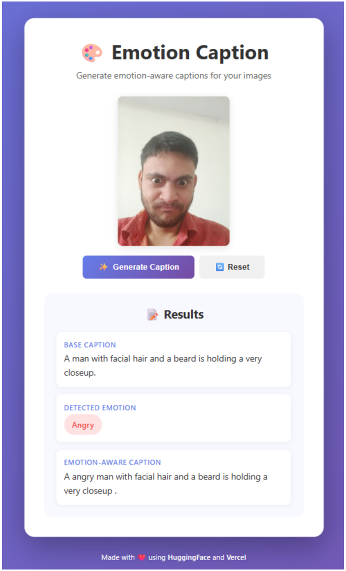
Emotion-Aware Caption Detected: Neutral

"A young **calm** girl with a swimsuit and goggles comes her head from above a pool."

Live Web App — Demo Screenshot (Happy)



Live Web App — Demo Screenshot (Angry)



Deployment Architecture

Backend:

- **FastAPI** (Python) on HuggingFace Spaces
- Docker container (Python 3.10-slim)
- Model downloaded from HuggingFace Hub on startup
- Endpoints: `GET /` (health check), `POST /api/caption`
- **URL:** `https://HimadriBiswas-emotion-caption-api.hf.space`

Frontend:

- **React 18** on Vercel
- Drag-and-drop image upload
- Displays: base caption, emotion badge, emotion-aware caption
- **URL:** `https://emotion-caption-frontend.vercel.app`

Deployment Challenges Solved:

- Model (1.91 GB) $>$ HuggingFace 1 GB limit $\rightarrow$ stored separately in HF Models Hub
- TensorFlow 2.13.0 compatibility pinned for DeepFace

Limitations & Future Work

Current Limitations:

- Adjective insertion can be grammatically awkward (e.g., plural nouns)
- Only dominant emotion from most prominent face is used
- Semantic mismatch: "disgusted person" vs. the intended meaning
- Little gibberish base captions on completely new images

Future Directions:

- **Multiple face** emotion handling per image
- **Semantic validators** for more natural adjective choice
- **Model quantization** to reduce size and cold-start time
- **Video captioning** with temporal emotion tracking

References

1. Mishra, S. et al., *Image Caption Generation using Vision Transformer and GPT Architecture*. 2024 2nd International Conference on Advancement in Computation & Computer Technologies (InCACCT). DOI: https://doi.org/10.1109/InCACCT61598.2024.10551257

2. Venkatesan, R.; Shirly, S.; Selvarathi, M.; Jebaseeli, T.J., *Human Emotion Detection Using DeepFace and Artificial Intelligence*. *Engineering Proceedings* 2023, 59(37). DOI: https://doi.org/10.3390/engproc2023059037

Conclusion

Summary of Achievements

- Built a complete **3-stage emotion-aware image captioning pipeline**
- Achieved **METEOR 44.18%** on Flickr30k dataset
- Successfully integrated **7-category emotion detection** via DeepFace
- Deployed as a **live web application** at zero cost
- Full open codebase on Kaggle Notebook

Key Takeaway:

Combining pre-trained ViT + GPT-2 with rule-based emotion integration provides a pragmatic, lightweight approach to **emotionally-aware image descriptions**.

This bridges the gap between *what* machines see and *how* humans feel.

Thank You!

Live Demo: `https://emotion-caption-frontend.vercel.app`

Team: TeamML Newbies